

Student

Week 2 gave me the basics - facts, dimensions, star schema, grain. Now I want to go deeper. What makes fact table design the hardest part of dimensional modeling?

Claude • AI Assistant

Because every downstream decision - query performance, aggregation correctness, BI tool behavior - flows from two choices you make when you design a fact table: the **grain** (what exactly one row represents) and the **measure type** (how the numbers in that row behave mathematically). Get either wrong and you'll produce reports that are technically correct SQL producing factually wrong business answers.

This chapter makes you fluent in both. Five precision subsections:

1. **Fact table types** - the three grains: transaction, periodic snapshot, accumulating snapshot.
2. **Additive, semi-additive, and non-additive facts** - the math that governs aggregation.
3. **Factless fact tables** - capturing events and coverage with no measures.
4. **Degenerate dimensions** - dimension attributes that live in the fact table.
5. **Junk dimensions** - consolidating low-cardinality flags without polluting the fact.

CONCEPT CARD

The grain statement - the most important sentence you will ever write for a fact table:

1. **What is it?** A single, precise English sentence declaring what one row in the fact table represents.
2. **Why it matters:** Every column in the fact table must be consistent with the grain. Every foreign key must point to a dimension at or above that grain. Every measure must make sense for that grain.
3. **The test:** Read the grain statement, then read each column. Ask: 'For this one row (this one [grain event]), does this column make sense?' If no - wrong grain or wrong column.
4. **Retail example:** 'One row per line item on a sales transaction.' Not 'one row per transaction' (too coarse) and not 'one row per product in the store' (different fact entirely).

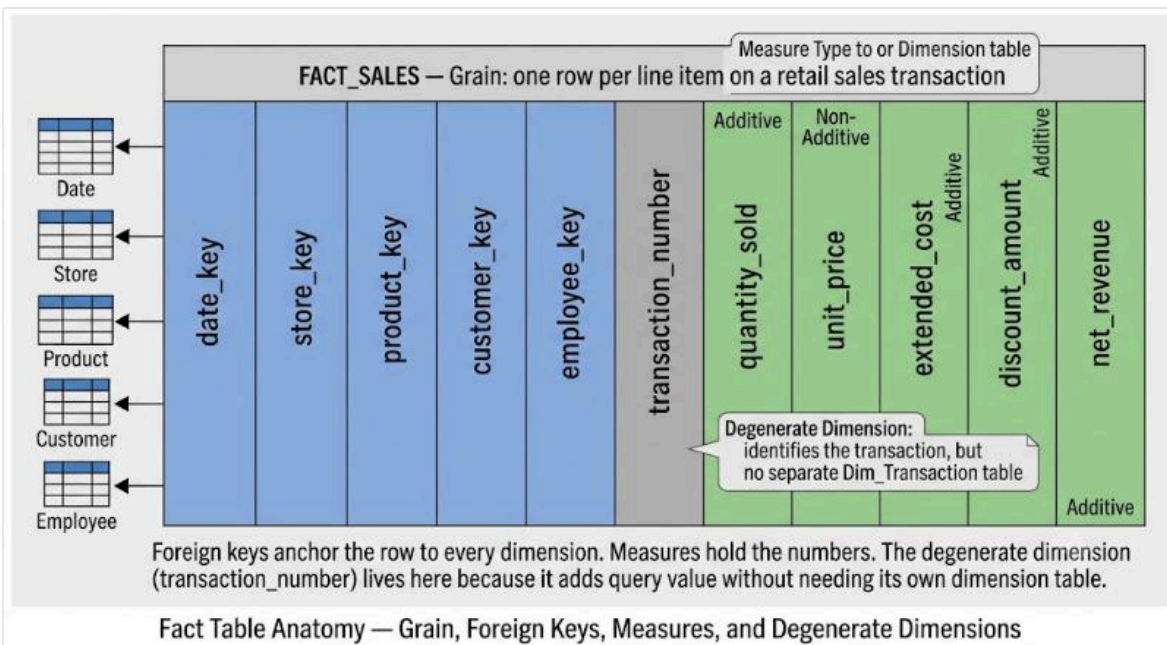
SUMMARY CARD

By the end of this chapter you will be able to:

- Choose the correct fact table type (transaction, periodic snapshot, accumulating snapshot) for any business process.
- Classify every measure as additive, semi-additive, or non-additive - and explain what BI tools must do differently for each.
- Design a factless fact table for event tracking and coverage problems.
- Recognize when a dimension attribute correctly belongs in the fact table (degenerate dimension) vs when it is a modeling mistake.
- Build a junk dimension and justify when it is the right solution.

🔮 AI DEEP DIVE PROMPT

Warm-up (paste into ChatGPT or Claude): "I am designing a fact table for a retail chain's point-of-sale transactions. Write the grain statement, identify three additive measures and one semi-additive measure, and explain why the invoice number would be a degenerate dimension rather than a foreign key to a separate Invoice dimension table."



Fact Table Anatomy - Grain, Foreign Keys, Measures, and Degenerate Dimensions

Three Fact Table Types - Grain, Update Behavior, and Best Use Cases

Type	Grain (one row per...)	Row inserted?	Row updated?	Best for
Transaction fact	Event (sale, click, payment)	At event time	Never	High-volume events; precise slice-and-dice

Type	Grain (one row per...)	Row inserted?	Row updated?	Best for
Periodic snapshot fact	Entity × time period (account × month-end)	Each period close	Never	Status tracking; inventory levels; balances
Accumulating snapshot fact	Process instance (order from placed to shipped)	At creation	Yes - at each milestone	Pipeline tracking; SLA measurement

sql – DDL for all three fact table types – retail context

```
-- =====
-- TRANSACTION FACT TABLE
-- Grain: one row per line item on a sales transaction
-- =====

CREATE TABLE fact_sales_transaction (
    date_key          INT      NOT NULL,  -- FK → dim_date
    store_key         INT      NOT NULL,  -- FK → dim_store
    product_key       INT      NOT NULL,  -- FK → dim_product
    customer_key      INT      NOT NULL,  -- FK → dim_customer
    transaction_number VARCHAR(20),      -- DEGENERATE DIMENSION (no FK)
    quantity_sold     INT      NOT NULL,  -- additive
    unit_price        DECIMAL(10,2),     -- NON-additive (never SUM this)
    extended_cost     DECIMAL(12,2),     -- additive
    discount_amount   DECIMAL(10,2),     -- additive
    net_revenue       DECIMAL(12,2),     -- additive
    PRIMARY KEY (date_key, store_key, product_key, transaction_number)
);

-- =====
-- PERIODIC SNAPSHOT FACT TABLE
-- Grain: one row per store × product × calendar month
-- =====

CREATE TABLE fact_inventory_snapshot (
    month_end_date_key INT      NOT NULL,  -- FK → dim_date (month-end only)
    store_key         INT      NOT NULL,  -- FK → dim_store
    product_key       INT      NOT NULL,  -- FK → dim_product
    units_on_hand     INT,              -- SEMI-ADDITIVE (sum by
store/product, not time)
    units_on_order    INT,              -- SEMI-ADDITIVE
    reorder_point     INT,              -- non-additive (policy
threshold)
```

```

PRIMARY KEY (month_end_date_key, store_key, product_key)
);

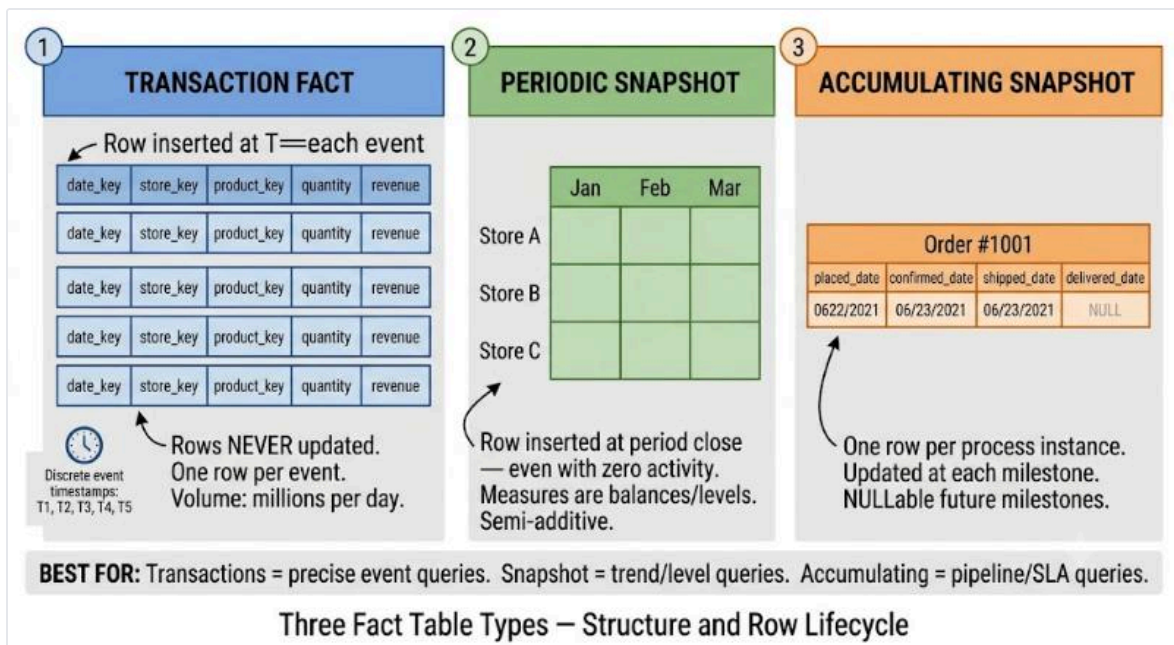
-- =====
-- ACCUMULATING SNAPSHOT FACT TABLE
-- Grain: one row per customer order (updated at each milestone)
-- =====

CREATE TABLE fact_order_fulfillment (
    order_key          INT      NOT NULL,  -- FK → dim_order
    customer_key       INT      NOT NULL,  -- FK → dim_customer
    order_placed_date_key INT,           -- FK → dim_date
    order_confirmed_date_key INT,       -- FK → dim_date (NULLable
until reached)
    shipped_date_key    INT,           -- FK → dim_date (NULLable)
    delivered_date_key INT,           -- FK → dim_date (NULLable)
    days_to_ship       INT,           -- lag: placed → shipped
    (derived)
    days_to_deliver    INT,           -- lag: placed → delivered
    order_total        DECIMAL(12,2), -- additive
    items_ordered      INT,           -- additive
    PRIMARY KEY (order_key)
);

```

FACT TABLE TYPES — TRANSACTION, PERIODIC SNAPSHOT, ACCUMULATING SNAPSHOT

Claude • AI Assistant



Three Fact Table Types - Structure and Row Lifecycle

Student

I know a fact table holds measures, but when should I use which type?

Claude • AI Assistant

The three fact table types answer three different business questions. Choose the type that matches the *nature of the business process* you are modeling - not the reporting preference. The grain drives everything else.

CONCEPT CARD

Transaction Fact Table - the workhorse:

1. **Grain** - one row per business event (a sale, a payment, a click, a return).
2. **When a row is inserted** - exactly once, at the moment the event occurs. Never updated.
3. **Measures** - precise event-level numbers: quantity, price, cost, discount.
4. **Volume** - typically the largest fact table by row count. A supermarket chain may have 50 million rows per day.
5. **Best for** - 'show me every transaction'; 'what sold yesterday at each store'; any question requiring event-level precision.

CONCEPT CARD

Periodic Snapshot Fact Table - the status tracker:

1. **Grain** - one row per entity (account, product, store) per time period (day-end, month-end, quarter-end).
2. **When a row is inserted** - at the close of each period, regardless of whether activity occurred. A product with zero sales still gets a row showing zero inventory movement.
3. **Measures** - balances and levels: inventory on hand, account balance, headcount.
4. **Key insight** - even if nothing happened in the period, the row exists. This is what makes trend queries possible: `SELECT month_key, SUM(units_on_hand)` across 24 months without any event to hang the data on.
5. **Best for** - 'what is our inventory level at month-end over the past two years?'; 'how has the account balance trended?'

CONCEPT CARD

Accumulating Snapshot Fact Table - the pipeline tracker:

1. **Grain** - one row per process instance (one order, one claim, one loan application).
2. **When a row is inserted** - at process creation. **When updated** - each time a milestone is reached (order confirmed, shipped, delivered).
3. **Measures** - lag days between milestones, final totals.
4. **Multiple date foreign keys** - the defining structural feature: *orderplaceddatekey*, *confirmeddatekey*, *shippeddatekey*, *delivereddate_key* - each pointing to the same date dimension.
5. **NULL-able milestones** - date keys for milestones not yet reached are NULL; filled in as the process progresses.
6. **Best for** - 'what percentage of orders ship within 2 days?'; 'where in the pipeline are orders stuck?'

🔗 ANALOGY

Think of the three types as three kinds of photography. Transaction fact = **continuous video recording** - every event captured at the instant it happens.

Periodic snapshot = **time-lapse photography** - a still frame taken at regular intervals whether or not anything moved. Accumulating snapshot = **a single photo with multiple exposures** - one image per subject, but each milestone adds another layer to the same image.

⚠️ WARNING

The most common grain mistake: mixing a transaction fact with a periodic snapshot in the same table. 'Revenue per day per store' (transaction-derived) and 'inventory level at day-end' (snapshot) have fundamentally different update behaviors and should never share a fact table, even though both have a date key and a store key.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Building an accumulating snapshot for a high-volume transaction process (1 million orders per day). Each order row is updated multiple times - mass updates on hundreds of millions of rows are operationally painful.

Correct Way: Use transaction facts for high-volume events; reserve accumulating snapshots for lower-volume pipeline tracking where row-by-row update is manageable.

EXAM TIP CARD

Interview classic: "You need to track inventory levels at month-end across 500 stores and 10,000 products for the past three years. Which fact table type and why?" Winning answer: *Periodic snapshot.

Grain: one row per store × product × month-end. A row exists even if no sales occurred - critical for accurate trend analysis.

Measures are semi-additive (sum across stores and products but not across months).

Transaction facts can't answer this without complex MAX/LAG gymnastics."*

🔥 EXERCISE

For each scenario, name the fact table type and write the grain statement: (a) recording every product return at a retail store, (b) tracking a mortgage loan from application to closing, (c) measuring warehouse stock levels at each week-end.

🧙 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "A hospital wants to track patient visits (every encounter logged), bed occupancy at midnight each night, and patient journeys through admission → diagnosis → treatment → discharge. Design three separate fact tables - one for each business process - specifying type, grain, sample measures, and why the type fits the process."

ADDITIVE, SEMI-ADDITIVE, AND NON-ADDITIVE FACTS

Claude • AI Assistant

Additive vs Semi-Additive vs Non-Additive — The Aggregation Safety Matrix

	Sum by Product Dimension	Sum by Store Dimension	Sum by Time Dimension
net_revenue (additive)	✓	✓	✓ ✓
units_on_hand (semi-additive)	✓	✓	✗ Use AVG or LAST instead
unit_price (non-additive)	✗	✗	✗ Derive at query time: SUM(revenue)/SUM(qty)

If a column is a ratio or percent, store the additive numerator and denominator separately. Never store the derived ratio in the fact table as the only representation.

WRONG: `AVG(unit_price)`.
RIGHT: `SUM(net_revenue) /
NULLIF(SUM(quantity_sold), 0) AS
avg_unit_price`.

Additive vs Semi-Additive vs Non-Additive — The Aggregation Safety Matrix

Additive vs Semi-Additive vs Non-Additive - The Aggregation Safety Matrix

sql – Correct aggregation patterns for each measure type

```
-- =====
-- ADDITIVE: SUM freely in any direction
-- =====
SELECT
    s.store_name,
    d.year_number,
    SUM(f.net_revenue)      AS total_revenue,  -- ✓ correct
    SUM(f.quantity_sold)   AS total_units,     -- ✓ correct
    SUM(f.discount_amount) AS total_discounts -- ✓ correct
FROM fact_sales_transaction f
JOIN dim_store s ON f.store_key = s.store_key
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY s.store_name, d.year_number;

-- =====
-- SEMI-ADDITIVE (inventory): sum across space, not time
-- =====
-- WRONG – sums 12 month-end balances (meaningless total)
SELECT SUM(units_on_hand) FROM fact_inventory_snapshot;

-- RIGHT – inventory at one specific month-end, by store
SELECT
    s.store_name,
    SUM(i.units_on_hand) AS end_of_month_units -- ✓ one period only
FROM fact_inventory_snapshot i
```

```

JOIN dim_store s ON i.store_key      = s.store_key
WHERE i.month_end_date_key = 20241231      -- single period
GROUP BY s.store_name;

-- RIGHT – average monthly inventory for trend
SELECT
    d.year_month,
    AVG(i.units_on_hand) AS avg_monthly_inventory -- ✓ avg across periods
FROM fact_inventory_snapshot i
JOIN dim_date s ON i.month_end_date_key = s.date_key
GROUP BY d.year_month;

-- =====
-- NON-ADDITIVE (unit_price): derive from additives
-- =====
-- WRONG – averaging unit prices ignores volume
SELECT AVG(unit_price) AS avg_price FROM fact_sales_transaction;

-- RIGHT – volume-weighted average price
SELECT
    p.product_name,
    SUM(f.net_revenue) / NULLIF(SUM(f.quantity_sold), 0) AS
volume_weighted_price
FROM fact_sales_transaction f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.product_name;

```

Student

Why does it matter whether a measure is additive? My BI tool just sums everything.

Claude • AI Assistant

That is exactly the problem. Your BI tool DOES just sum everything - unless you stop it. When a semi-additive or non-additive measure gets summed across the wrong dimension, the result is a number that looks like valid output but is factually wrong. Misclassified measures are silent data quality errors that reach executive dashboards.

CONCEPT CARD

Additive measures - safest to aggregate:

1. **Definition** - can be summed across ALL dimensions in the model. Revenue, cost, quantity, discount amount.
2. **BI tool behavior** - default SUM works correctly in all directions: by store, by product, by date, by store+product, total.
3. **The test** - 'Does summing this column across any combination of dimensions produce a meaningful number?' If yes: additive.
4. **Example** - `net_revenue` : SUM by store = store revenue ✓. SUM by product = product revenue ✓. SUM by date = daily revenue ✓. Grand total = total revenue ✓. All correct.

CONCEPT CARD

Semi-additive measures - the most mishandled:

1. **Definition** - can be summed across SOME dimensions but NOT across time.
2. **Classic examples** - account balance, inventory on hand, headcount, portfolio value.
3. **The problem** - `SUM(inventory_on_hand)` across stores is meaningful (total inventory across all stores). But `SUM(inventory_on_hand)` across 12 months produces a nonsense number - you don't have 12 months of inventory stacked.
4. **Correct time aggregation** - `AVG` (for trend), `MAX`, `MIN`, or `LAST` (most recent period). Never SUM across time periods.
5. **BI tool fix** - in Power BI, configure the measure as 'Last Non-Empty'. In Tableau, build a calculated field using LOD expressions. Never leave default SUM.

CONCEPT CARD

Non-additive measures - never sum, never average naively:

1. **Definition** - cannot be meaningfully summed or averaged across ANY dimension.
2. **Classic examples** - *unitprice*, *profitmarginpercent*, *inventoryturnratio*, *exchangerate*.
3. **The problem** - `SUM(unit_price)` across 1,000 transactions produces a number with no business meaning. `AVG(unit_price)` across SKUs ignores volume weighting.
4. **Correct approach** - store the underlying additive components (revenue, cost, quantity) and derive the ratio at query time: `SUM(net_revenue) / SUM(quantity_sold)` gives a volume-weighted average price.
5. **Design rule** - if a column is a ratio or percentage, store the numerator and denominator as separate additive measures. Derive the ratio in the query.

EXAMPLE CARD

Inventory scenario - why semi-additive matters:

1. A retailer has 50 stores. At month-end, each store's `units_on_hand` is recorded in the periodic snapshot.
2. `SUM(units_on_hand) GROUP BY store_key` → total units across all stores for that month.
Correct - summing across store dimension.

3. `SUM(units_on_hand) GROUP BY month_key` → sum of January + February + March on-hand counts. **Wrong - not 3 months of inventory. Should use LAST or AVG.**
4. `SUM(units_on_hand)` overall → wrong. `AVG(units_on_hand)` by month → average monthly inventory level. Meaningful.

🔗 ANALOGY

Think of it this way. **Additive** = a bag of marbles.

Any subset sums correctly. **Semi-additive** = a bank balance.

Add balances across accounts = total deposits. Add January balance + February balance = meaningless.

Non-additive = a temperature reading. `SUM(temperature)` across sensors or days has no physical meaning.

Always restate as a derived figure.

⚠️ WARNING

The most dangerous non-additive measure is **margin percentage stored directly in the fact table**. Teams that store `margin_pct = (revenue - cost) / revenue` as a column almost always SUM or AVG it somewhere incorrectly.

Always store `net_revenue` and `cost_of_goods_sold` separately; compute `margin_pct` in the query.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: `SELECT month_name, SUM(account_balance) FROM fact_account_snapshot GROUP BY month_name` - sums balances across 12 months, producing 12× the actual balance.

Correct Way: `SELECT month_name, SUM(account_balance) FROM fact_account_snapshot WHERE month_end_date_key = :target_month GROUP BY month_name` - one period at a time. Or for trend: use `MAX(account_balance)` for period-end balance, or `AVG(account_balance)` for a period-average metric.

EXAM TIP CARD

Interview classic: "Give me an example of a semi-additive measure and explain what happens if you SUM it incorrectly." Winning answer: "Account balance in a banking warehouse. Stored in a monthly periodic snapshot.

SUMming across accounts is correct - total deposits in the bank. SUMming across months gives January's balance + February's balance + ... - a number that exists nowhere in reality.

BI tools must use LAST NON-EMPTY or point-in-time filtering for time-axis aggregation."

🔥 EXERCISE

Classify each column from a hotel fact table as additive, semi-additive, or non-additive: (a) *roomsrevenue*, (b) *roomsoccupied* (nightly snapshot), (c) *averagedailyrate*, (d) *roomcost*, (e) *occupancypercentage*, (f) *booking_count*.

🔮 AI DEEP DIVE PROMPT

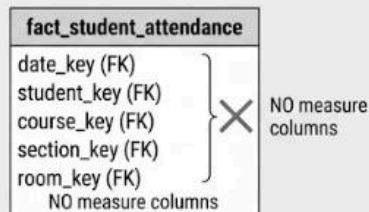
Paste into ChatGPT or Claude: *"I have a financial services data warehouse with a periodic snapshot fact table containing: accountbalance, interestaccrued, interestrate, transactioncount, and dayspastdue. Classify each measure as additive, semi-additive, or non-additive.*

For each semi-additive and non-additive measure, write the correct SQL aggregation pattern."

FACTLESS FACT TABLES — EVENTS AND COVERAGE WITHOUT MEASURES

Claude • AI Assistant

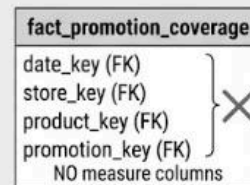
EVENT-CAPTURING FACTLESS FACT — fact_student_attendance



Count of rows = number of attendances.
COUNT(*) is the measure.

```
SELECT student_key,
       COUNT(*) AS sessions_attended
FROM fact_student_attendance
GROUP BY student_key.
```

COVERAGE FACTLESS FACT — fact_promotion_coverage



fact_promotion_coverage
(what SHOULD have happened)

Promoted but
did NOT sell —
only visible with
coverage table!

LEFT JOIN

fact_sales_transaction

fact_sales_transaction
(what DID happen)

Without the coverage table, the absence of sales is invisible
— zero rows in the transaction fact means zero information.
Coverage tables make the denominator explicit.

Factless Fact Tables — Event Capture vs Coverage Pattern

Factless Fact Tables - Event Capture vs Coverage Pattern

sql — Factless fact table DDL and the coverage query pattern

```
-- =====
-- FACTLESS FACT: promotion coverage
-- Grain: one row per date x store x product that a promotion was active
-- =====
```

```

CREATE TABLE fact_promotion_coverage (
    date_key          INT NOT NULL,  -- FK → dim_date
    store_key         INT NOT NULL,  -- FK → dim_store
    product_key       INT NOT NULL,  -- FK → dim_product
    promotion_key     INT NOT NULL,  -- FK → dim_promotion
    -- NO measure columns – the row's existence is the fact
    PRIMARY KEY (date_key, store_key, product_key, promotion_key)
);

-- =====
-- COVERAGE QUERY: promoted products that did NOT sell
-- =====

SELECT
    promo.promotion_name,
    s.store_name,
    p.product_name,
    d.calendar_date
FROM fact_promotion_coverage  cov
JOIN dim_promotion            promo ON cov.promotion_key =
promo.promotion_key
JOIN dim_store                s      ON cov.store_key    = s.store_key
JOIN dim_product              p      ON cov.product_key  = p.product_key
JOIN dim_date                 d      ON cov.date_key     = d.date_key
-- LEFT JOIN to sales: no matching sale = promoted but unsold
LEFT JOIN fact_sales_transaction sale
    ON sale.date_key  = cov.date_key
   AND sale.store_key = cov.store_key
   AND sale.product_key = cov.product_key
WHERE sale.date_key IS NULL  -- the key indicator: no sale row matched
ORDER BY promo.promotion_name, s.store_name, p.product_name;

-- =====
-- CONVERSION RATE: % of promoted store-days with at least one sale
-- =====

SELECT
    promo.promotion_name,
    COUNT(DISTINCT cov.date_key || '-' || cov.store_key)          AS
coverage_store_days,
    COUNT(DISTINCT sale.date_key || '-' || sale.store_key)        AS
selling_store_days,
    ROUND(100.0 * COUNT(DISTINCT sale.date_key || '-' || sale.store_key)
          / NULLIF(COUNT(DISTINCT cov.date_key || '-' || cov.store_key),
0), 1) AS conversion_pct
FROM fact_promotion_coverage cov
JOIN dim_promotion            promo ON cov.promotion_key = promo.promotion_key
LEFT JOIN fact_sales_transaction sale

```

```

ON sale.date_key    = cov.date_key
AND sale.store_key  = cov.store_key
AND sale.product_key = cov.product_key
GROUP BY promo.promotion_name;

```

Student

What is a factless fact table? If there are no measures, what is the point?

Claude • AI Assistant

The point is that **the event itself - or its absence - is the information**. Not every business fact has a natural numeric measure. A student attending class, a promotion being active in a store, or a website page being visited are meaningful events even if there is no dollar amount or quantity attached.

A factless fact table captures these events as rows, with only foreign keys and no measure columns.

CONCEPT CARD

Two flavors of factless fact tables:

1. **Event-capturing factless facts** - records that something happened. Student attended lecture. Customer visited product page. Doctor ordered a test. The count of rows IS the measure.
2. **Coverage factless facts** - records what COULD have happened. Promotion X was active in Store Y on Date Z. This is the denominator that makes 'promotion effectiveness' queries possible. Without coverage, you cannot compute 'which promoted products did NOT sell?'

CONCEPT CARD

The 'count the rows' pattern:

1. The standard measure for a factless fact table is `COUNT(*)` or `COUNT(DISTINCT key)`.
2. Example: `SELECT product_key, COUNT(*) AS page_views FROM fact_product_page_view GROUP BY product_key` - each row is one page view.
3. No quantity column needed; the row's existence encodes the event.
4. You can still join to dimensions to slice: by date, by customer segment, by device type.

CONCEPT CARD

The coverage query pattern - the killer use case:

1. **Fact table 1** - `fact_promotion_coverage(date_key, store_key, product_key)` : every combination where a promotion was scheduled.
2. **Fact table 2** - `fact_sales_transaction(date_key, store_key, product_key, ...)` : every actual sale.
3. **The business question** - 'Which promoted products failed to sell in the stores where they were promoted?'
4. **SQL pattern** - `LEFT JOIN` or `NOT EXISTS` between coverage and transaction fact, filtered to promoted dates.
5. **Without the coverage table** - this question cannot be answered. You have no list of 'what should have happened.'

EXAMPLE CARD

Student attendance tracking:

1. `fact_class_enrollment(date_key, student_key, course_key, section_key)` - no measures.
2. Enrolled students who skipped a class: `LEFT JOIN` scheduled sessions against attendance rows; `WHERE attendance row IS NULL`.
3. Attendance rate: `COUNT(attendance rows) / COUNT(enrollment rows)` - both from factless facts.
4. At-risk students: those below 75% attendance - entirely derivable from row counts with no stored measures.

🔗 ANALOGY

A factless fact table is like a **hospital visitor log**. The log records who came in, when, and for whom.

There is no 'amount' column - the fact is the visit itself. Investigators use the log to answer: 'Who visited patient X last Tuesday?' and 'Who on the visitor list never showed up?' Both questions come from counting and joining rows, not summing a measure.

⚠️ WARNING

Do not add a dummy measure column (e.g., `event_count INT DEFAULT 1`) just to make the table look like a 'normal' fact table. It is unnecessary, adds storage, and confuses future developers. The count of rows is naturally obtained with `COUNT(*)`.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Answering 'which promotions had no sales?' by querying only the transaction fact table. If a promotion had zero sales, it has zero rows in the transaction fact - it simply doesn't appear. You cannot see the absence.

Correct Way: Build a promotion-coverage factless fact table. LEFT JOIN from coverage to transactions. WHERE transaction row IS NULL gives you the promoted products that never sold.

EXAM TIP CARD

Interview classic: "How would you determine which stores failed to sell a promoted product during a promotional period?" Winning answer: "Build a factless coverage fact table with grain: one row per promotion × store × product × date that the promotion was scheduled. LEFT JOIN from this coverage table to the sales transaction fact on store, product, and date. Filter WHERE the transaction fact's join columns are NULL. The non-matching rows are the promoted combinations with zero sales."

🔥 EXERCISE

Design a factless fact table for a university's exam scheduling system: one row per student × exam × room × date. Write the SQL to find students who were registered for an exam but have no attendance record in the system.

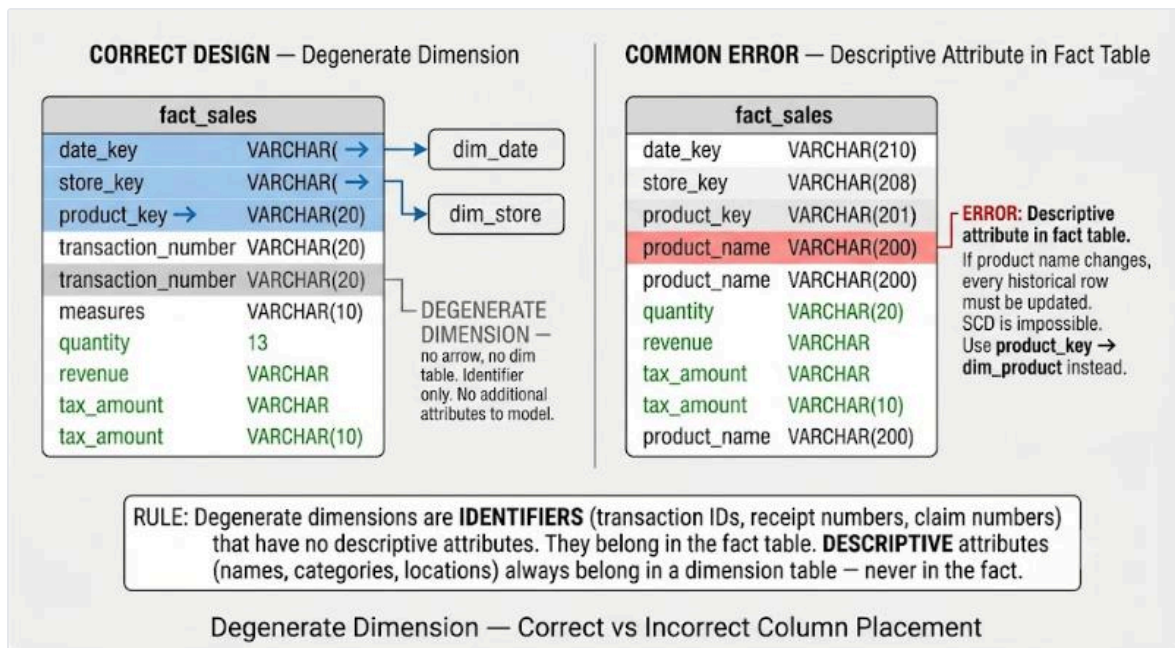
🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "Design a factless fact table for a retail promotional coverage system. Include the grain statement, all foreign keys, and write SQL to answer: (a) total store-days a promotion was active, (b) which promoted products sold in less than 30% of the stores where they were promoted, (c) the coverage-to-sales conversion rate by promotion."

DEGENERATE DIMENSIONS — DIMENSION ATTRIBUTES IN THE FACT TABLE



Claude • AI Assistant



Degenerate Dimension - Correct vs Incorrect Column Placement

Student

What is a degenerate dimension? Isn't it wrong to put dimension data in a fact table?

Claude • AI Assistant

It is absolutely NOT wrong - it is the correct design for a specific class of attribute. A **degenerate dimension** is a dimension key stored in the fact table that has no corresponding dimension table.

The attribute has identifying value for queries but no additional descriptive columns to justify building a separate table.

CONCEPT CARD

The canonical degenerate dimension:

1. **Transaction number / invoice number / order number** - identifies the source transaction.
2. **Why it belongs in the fact table** - it is used in queries ('show me all line items on invoice INV-20241105-0042'), but there is nothing more to know about an invoice number itself beyond its existence. No descriptive attributes.
3. **Why NOT a separate Dim_Invoice table** - it would have one column (invoice_number), serve only as a look-up back to itself, and add a join with no benefit.

4. **How to recognize it** - the attribute acts like a dimension key (it identifies a business entity) but has no hierarchy, no descriptive attributes, and no associated lookup table.

CONCEPT CARD

Common degenerate dimensions in practice:

1. **Retail** - *transactionnumber, receiptnumber, POSTerminalid*.
2. **E-commerce** - *orderid, sessionid, cart_id*.
3. **Finance** - *checknumber, wiretransferid, journalentry_number*.
4. **Healthcare** - *visitnumber, claimnumber, prescription_id*.
5. **Logistics** - *shipmentnumber, trackingid, billoflading_number*.

CONCEPT CARD

How degenerate dimensions are used in queries:

1. **Grouping a transaction** - `GROUP BY transaction_number` to reassemble a receipt.
2. **Filtering by identifier** - `WHERE transaction_number = 'INV-042'` to find specific transactions.
3. **Linking two fact tables** - the transaction number in a sales fact can match a returns fact to link a return to its originating sale.
4. **They do not join to a dimension table** - they are simply stored as a VARCHAR or INT in the fact, with no FK constraint to a dimension.

🔗 ANALOGY

A degenerate dimension is like a **driver's license number on a medical form**. The number identifies the patient uniquely for this visit.

But the clinic doesn't maintain a separate 'Driver's License' dimension table with rows for every license. The number lives on the record (fact) because it has identifying value - not because there's a rich profile behind it.

⚠️ WARNING

Do not confuse a degenerate dimension with a modeling error. A modeling error is storing a dimension ATTRIBUTE (like `product_name` or `store_city`) in the fact table instead of a foreign key.

A degenerate dimension is storing a dimension KEY (an identifier) when the referenced entity genuinely has no useful descriptive attributes to maintain in a separate table.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Storing `product_name VARCHAR(200)` directly in the fact table instead of `product_key INT`. This is NOT a degenerate dimension - it is denormalization that breaks SCD and makes updates impossible.

Correct Way: `product_key` in the fact table → `dim_product.product_key` . Degenerate dimensions are IDENTIFIERS (transaction numbers, receipt IDs) - not descriptive attributes.

EXAM TIP CARD

Interview classic: "What is a degenerate dimension and how is it different from just putting a column in the wrong place?" Winning answer: * "A degenerate dimension is an identifier attribute - like an order number or invoice ID - stored in the fact table with no corresponding dimension table, because there are no descriptive attributes that would justify a separate table. It is correct design.

By contrast, storing a dimension's descriptive attributes (product name, store city) directly in the fact table is a denormalization error that breaks slowly changing dimension management and creates update anomalies."*

🔥 EXERCISE

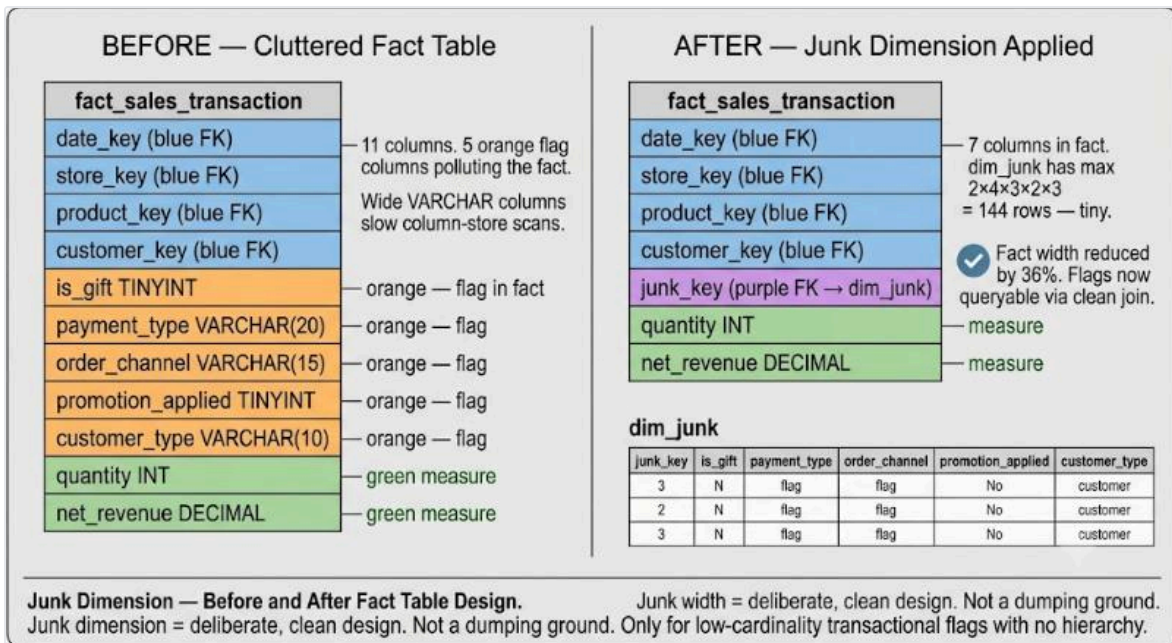
Examine this fact table column list: (`sale_date_key`, `store_key`, `product_key`, `cashier_employee_key`, `register_number`, `receipt_number`, `quantity`, `revenue`, `tax_amount`) . Identify: (a) the foreign keys, (b) the additive measures, (c) any degenerate dimensions - and justify each classification.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: "For a healthcare claims fact table, identify five degenerate dimensions. For each, explain why it belongs in the fact table rather than a separate dimension, and write the SQL to use it in a query (e.g., retrieve all line items on a specific claim)."

JUNK DIMENSIONS — CONSOLIDATING LOW-CARDINALITY FLAGS

Claude • AI Assistant



Junk Dimension - Before and After Fact Table Design

sql – Junk dimension DDL and population – e-commerce example

```
-- =====
-- DIM_JUNK: transactional flags consolidated
-- =====

CREATE TABLE dim_order_junk (
    junk_key          INT          PRIMARY KEY,
    is_gift           CHAR(1)      NOT NULL,  -- Y / N
    payment_method    VARCHAR(15)  NOT NULL,  -- credit_card, debit_card,
UPI, COD
    delivery_type     VARCHAR(15)  NOT NULL,  -- standard, express,
same_day
    customer_type     VARCHAR(15)  NOT NULL   -- new, returning, VIP
);

-- =====
-- POPULATE: all possible combinations (Cartesian product)
-- 2 × 4 × 3 × 3 = 72 rows maximum
-- =====

INSERT INTO dim_order_junk (junk_key, is_gift, payment_method, delivery_type,
customer_type)
SELECT
    ROW_NUMBER() OVER (ORDER BY g.is_gift, p.payment_method,
                        d.delivery_type, c.customer_type) AS
junk_key,
    g.is_gift,
    p.payment_method,
```

```

        d.delivery_type,
        c.customer_type
FROM
    (VALUES ('Y'), ('N'))
g(is_gift)
CROSS JOIN
    (VALUES ('credit_card'), ('debit_card'), ('UPI'), ('COD'))
p(payment_method)
CROSS JOIN
    (VALUES ('standard'), ('express'), ('same_day'))
d(delivery_type)
CROSS JOIN
    (VALUES ('new'), ('returning'), ('VIP'))
c(customer_type);

-- =====
-- LOOKUP during ETL: get junk_key for this row's combination
-- =====
SELECT junk_key
FROM   dim_order_junk
WHERE  is_gift          = :src_is_gift
      AND payment_method = :src_payment_method
      AND delivery_type  = :src_delivery_type
      AND customer_type  = :src_customer_type;
-- Use the returned junk_key as the FK in fact_order_transactions

-- =====
-- QUERY: revenue by payment method and delivery type
-- =====
SELECT
    j.payment_method,
    j.delivery_type,
    SUM(f.net_revenue) AS total_revenue,
    COUNT(*)           AS order_count
FROM   fact_order_transactions f
JOIN   dim_order_junk         j ON f.junk_key = j.junk_key
GROUP BY j.payment_method, j.delivery_type
ORDER BY total_revenue DESC;

```

Student

I have five flag columns (*isgift*, *paymenttype*, *orderchannel*, *promotionapplied*, *customer_type*) in my fact table. Should they each be their own dimension?

Claude • AI Assistant

No - and they should not stay in the fact table as loose columns either. What you have described is the ideal use case for a **junk dimension**: a single small dimension table that bundles several low-cardinality flags and indicators, keeping the fact table narrow and clean.

CONCEPT CARD

What makes something a good junk dimension candidate:

1. **Low cardinality** - each flag has a small number of possible values (boolean, 3–10 options).
2. **No meaningful hierarchy** - the flags don't drill up or down into a parent-child structure.
3. **Mixed origin** - the flags come from different business processes but land on the same transaction.
4. **The alternative is bad** - five separate dimension tables for five boolean flags would each have 2 rows. Or five columns loose in the fact table polluting the schema.

CONCEPT CARD

Building the junk dimension - the Cartesian product approach:

1. **Enumerate all combinations** - if `is_gift` has 2 values and `payment_type` has 3 values and `order_channel` has 4 values: $2 \times 3 \times 4 = 24$ possible combinations.
2. **Pre-build the dimension** - `INSERT INTO dim_junk SELECT DISTINCT is_gift, payment_type, order_channel FROM source_transactions` . Or generate all combinations upfront.
3. **Assign a surrogate key** - `junk_key INT` is the foreign key stored in the fact table.
4. **Never grow unboundedly** - if the domain of flags is truly bounded (not millions of combinations), the junk dimension is small and stable. Beware: if cardinalities are high (e.g., `payment_type` has 500 values), a junk dimension may be wrong - use separate dimension tables.

CONCEPT CARD

Before and after - fact table impact:

1. **Before** - fact table has: `date_key, store_key, product_key, customer_key, is_gift TINYINT, payment_type VARCHAR(20), order_channel VARCHAR(15), promotion_applied TINYINT, customer_type VARCHAR(10), quantity INT, revenue DECIMAL` .
2. **After** - fact table has: `date_key, store_key, product_key, customer_key, junk_key, quantity INT, revenue DECIMAL` .

3. **Width reduction** - 11 columns → 7 columns. Fewer wide VARCHAR columns in the high-volume fact table. Better compression, faster scans.
4. **dim_junk** - small table: (junk_key, is_gift, payment_type, order_channel, promotion_applied, customer_type) . Queried by joining, not by scanning the fat fact.

🔗 ANALOGY

A junk dimension is like a **ticket stub drawer**. You have five different types of tickets (concert, train, museum, cinema, parking) that you collect but never need to look up individually in a detailed catalog.

You put them all in one drawer (junk dimension) labeled by the combination that was present on each day. The fact table just says 'combo 14 was used on this transaction' and you look up combo 14 in the drawer when you need to filter by type.

⚠️ WARNING

The name 'junk' is unfortunate - it implies disorder. In reality, a junk dimension is a **clean, deliberate design choice**.

Never use it as a catch-all for attributes that should have their own proper dimensions (like product category or store region). It is specifically for **transactional flags and indicators** that don't belong to a richer dimension hierarchy.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Adding a new column to the junk dimension every time a new flag appears in the source system. This requires rebuilding the entire Cartesian product, updating the surrogate key population logic, and potentially creating new junk_key values.

Design the junk dimension once with all known flags.

Correct Way: Identify ALL candidate flags before building. If new flags keep appearing, that signals the source process is unstable - address the source, not the dimension.

EXAM TIP CARD

Interview classic: "Why would you use a junk dimension instead of keeping the flags in the fact table?" Winning answer: "Three reasons: (1) Narrow fact tables compress and scan better - removing VARCHAR flag columns from a billion-row fact table has meaningful performance impact. (2) Conformed reuse - the junk dimension can be shared across multiple fact tables that share the same flag combinations.

(3) Manageability - filtering and grouping by flags becomes a join to a small, well-labeled dimension rather than repeated WHERE clauses on columns scattered across the fact."

🔥 EXERCISE

Build a junk dimension for an e-commerce order with these attributes: `is_gift` (Y/N) , `payment_method` (`credit_card`, `debit_card`, `UPI`, `COD`) , `delivery_type` (`standard`, `express`, `same_day`) , `customer_type` (`new`, `returning`, `VIP`) . (a) How many rows in the fully populated junk dimension?
(b) Write the DDL. (c) Write the INSERT to populate it.

🌟 AI DEEP DIVE PROMPT

Paste into ChatGPT or Claude: `""Design a junk dimension for a retail point-of-sale fact table. Include these flags: isloyaltymember (Y/N), paymentmethod (cash/card/UPI/voucher), storeformat (flagship/express/kiosk), promotionapplied (Y/N), isreturn_visit (Y/N).`

`Write the DDL, calculate the maximum possible rows, write the SQL to populate it from a staging table, and explain how the junk_key gets inserted into the fact table.""`

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

CHAPTER SUMMARY

You now have precise command of fact table design - the grain choice, the measure classification, and the five structural patterns that cover every real-world scenario.

- **FACT TABLE TYPES:** Transaction fact (grain: one row per event, never updated, highest volume), periodic snapshot (grain: entity × time period, inserted each period even with no activity, semi-additive measures), accumulating snapshot (grain: one row per process instance, updated at each milestone, multiple date foreign keys). Choose by nature of business process - not by reporting preference.
- **ADDITIVE, SEMI-ADDITIVE, NON-ADDITIVE:** Additive (SUM in any direction - revenue, quantity, cost). Semi-additive (SUM across space dimensions but not time - inventory levels, account balances - use AVG or LAST for time aggregation). Non-additive (never SUM - ratios, percentages, unit price - store additive numerator/denominator and derive at query time).
- **FACTLESS FACT TABLES:** Two patterns - event-capturing (one row per event, no measures, COUNT(*) is the fact) and coverage (records what could have happened, enabling LEFT JOIN to find absences). Coverage tables make 'denominator' queries possible - promoted-but-unsold, enrolled-but-absent.

- **DEGENERATE DIMENSIONS:** Identifiers (transaction number, invoice number, receipt ID) stored in the fact table with no corresponding dimension table. Correct when the identifier has identifying value but no descriptive attributes justifying a separate table. Not the same as storing descriptive attributes in the fact - that is an error.
- **JUNK DIMENSIONS:** Bundles several low-cardinality flags and indicators into one small dimension. Pre-built via Cartesian product. Narrows the fact table, improves compression, enables clean joins. Correct for transactional flags with no hierarchy and low cardinality. Not a catch-all for rich dimension attributes.

Claude • AI Assistant

With fact table mastery in hand, Chapter 2 turns to the dimension side: role-playing dimensions (one physical table, multiple meanings), bridge tables for many-to-many relationships, hierarchy patterns, the date dimension as the most important reused object in any warehouse, and mini-dimensions for managing rapidly-changing attributes without Type 2 row explosion.